Data manipulation in R

Part 2

Last session...

- The pipe (`|>`) chains commands together.
- `select()`, `filter()` and `arrange()` pick out and sort columns and rows.
- `mutate()` creates or modifies columns.
- `if_else()` and `case_when()` recode variables.
- `%in%` checks whether a value is one of a set.

```
starwars |>
    filter(species == "Human") |>
    select(name, height, mass) |>
    mutate(heavy = mass > 80) |>
    arrange(desc(height))
```

Session overview

Essential data manipulation tasks

a) ~~Select columns or rows~~
b) ~~Sorting a dataset~~
c) ~~Creating or modifying columns~~
d) Combining datasets
e) Reshaping a dataset
f) Grouping and summarising data

d) Combining datasets

Appending two data frames

id	age
001	45
002	33
003	63

+

height
1.87
1.43
1.68

⟶

id	age	height
001	45	1.87
002	33	1.43
003	63	1.68

d) Combining datasets

Appending two data frames

id	age
001	45
002	33
003	63

+

height
1.87
1.43
1.68

⟶

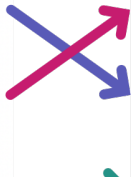
id	age	height
001	45	1.87
002	33	1.43
003	63	1.68

Merging or 'joining' two data frames

id	age
001	45
002	33
003	63

+

id	height
002	1.87
001	1.43
003	1.68



id	age	height
001	45	1.43
002	33	1.87
003	63	1.68

Append with `bind_rows` and `bind_cols`

```r
# 1. Select some columns
a <- starwars[, 2:4]
b <- starwars[, 10]
```

Append with `bind_rows` and `bind_cols`

```r
# 1. Select some columns
a <- starwars[, 2:4]
b <- starwars[, 10]
```

```
# a
  height  mass hair_color
   <int> <dbl> <chr>
1    172    77 blond
2    167    75 <NA>
3     96    32 <NA>
4    202   136 none
5    150    49 brown
6    178   120 brown, grey
7    165    75 brown
8     97    32 <NA>
# i 79 more rows
```

```
# b
  homeworld
  <chr>
1 Tatooine
2 Tatooine
3 Naboo
4 Tatooine
5 Alderaan
6 Tatooine
7 Tatooine
8 Tatooine
# i 79 more rows
```

```
# Bind them together
bind_cols(a, b)

# A tibble: 87 x 4
   height  mass hair_color  homeworld
    <int> <dbl> <chr>       <chr>
1     172    77 blond       Tatooine
2     167    75 <NA>        Tatooine
3      96    32 <NA>        Naboo
4     202   136 none        Tatooine
5     150    49 brown       Alderaan
6     178   120 brown, grey Tatooine
# i 81 more rows
```

```r
# For rows...
a <- starwars[1:5, ]
b <- starwars[20:30, ]

bind_rows(a, b)
```

```r
# For rows...
a <- starwars[1:5, ]
b <- starwars[20:30, ]

bind_rows(a, b)
```

These commands are replacements for `cbind` and `rbind` from base R.

Merging with `*_join`

- We merge to combine variables held in separate datasets based on one or more common keys.

Merging with `*_join`

- We merge to combine variables held in separate datasets based on one or more common keys.
- These operations are referred to as joins.

Merging with `*_join`

- We merge to combine variables held in separate datasets based on one or more common keys.
- These operations are referred to as joins.

x

1	x1
2	x2
3	x3

y

1	y1
2	y2
4	y3

Source: R for Data Science (Wickham and Grolemund, 2017)

Merging with `*_join`

- We merge to combine variables held in separate datasets based on one or more common keys.
- These operations are referred to as joins.

x

1	x1
2	x2
3	x3

y

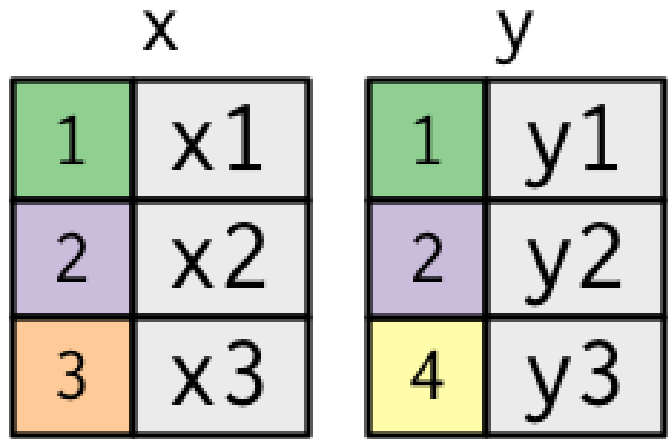
1	y1
2	y2
4	y3

- A join is a way of connecting each row in x to zero, one, or more rows in y.

Source: R for Data Science (Wickham and Grolemund, 2017)

Merging with `*_join`

- We merge to combine variables held in separate datasets based on one or more common keys.
- These operations are referred to as joins.



- A join is a way of connecting each row in x to zero, one, or more rows in y.
- The type of join we need depends on how many keys from x are also found in y.

Source: R for Data Science (Wickham and Grolemund, 2017)

We're going to focus on four types of join:

`inner_join`	matches pairs of observations whenever their keys are equal.
`left_join`	keeps all observations in `x`.
`right_join`	keeps all observations in `y`.
`full_join`	keeps all observations in `x` and `y`.

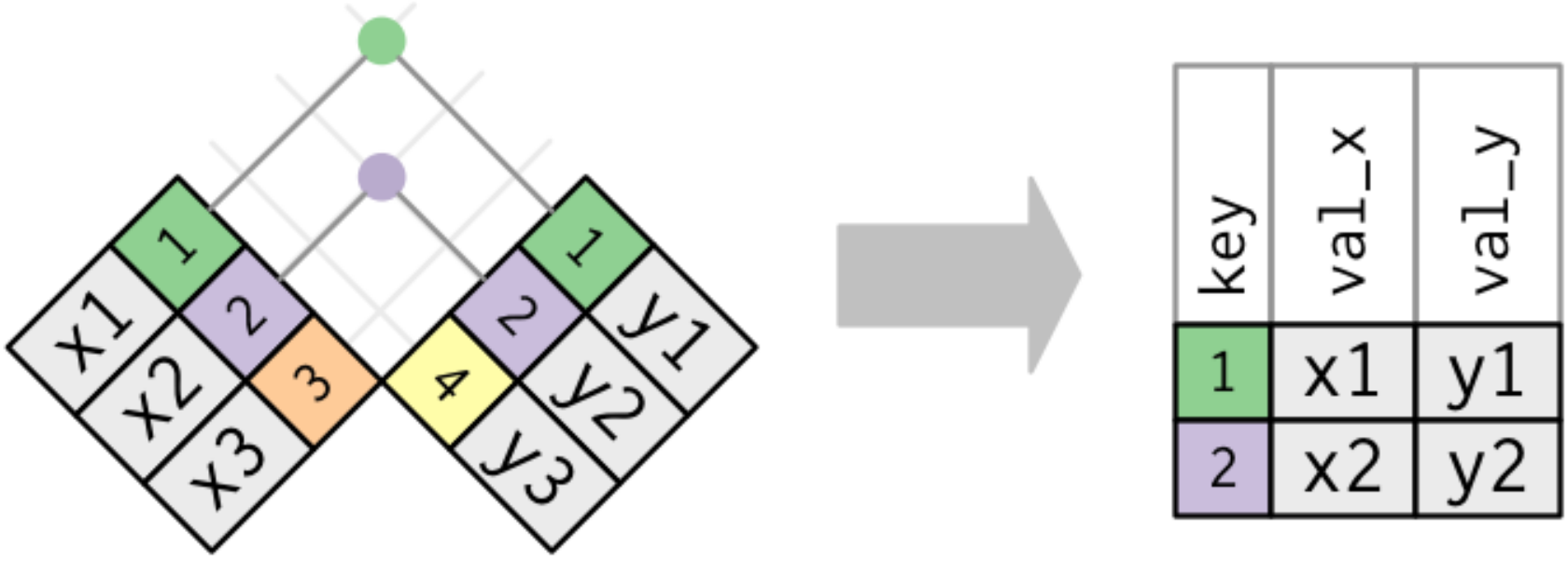
We're going to focus on four types of join:

`inner_join` matches pairs of observations whenever their keys are equal.

`left_join` keeps all observations in `x`.

`right_join` keeps all observations in `y`.

`full_join` keeps all observations in `x` and `y`.



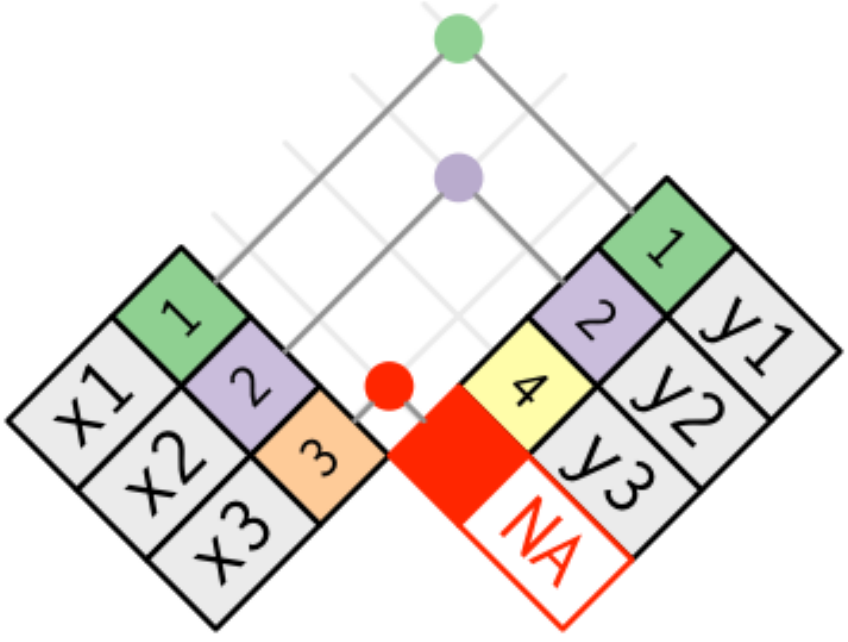
We're going to focus on four types of join:

`inner_join` matches pairs of observations whenever their keys are equal.

`left_join` keeps all observations in `x`.

`right_join` keeps all observations in `y`.

`full_join` keeps all observations in `x` and `y`.



key	val_x	val_y
1	x1	y1
2	x2	y2
3	x3	NA

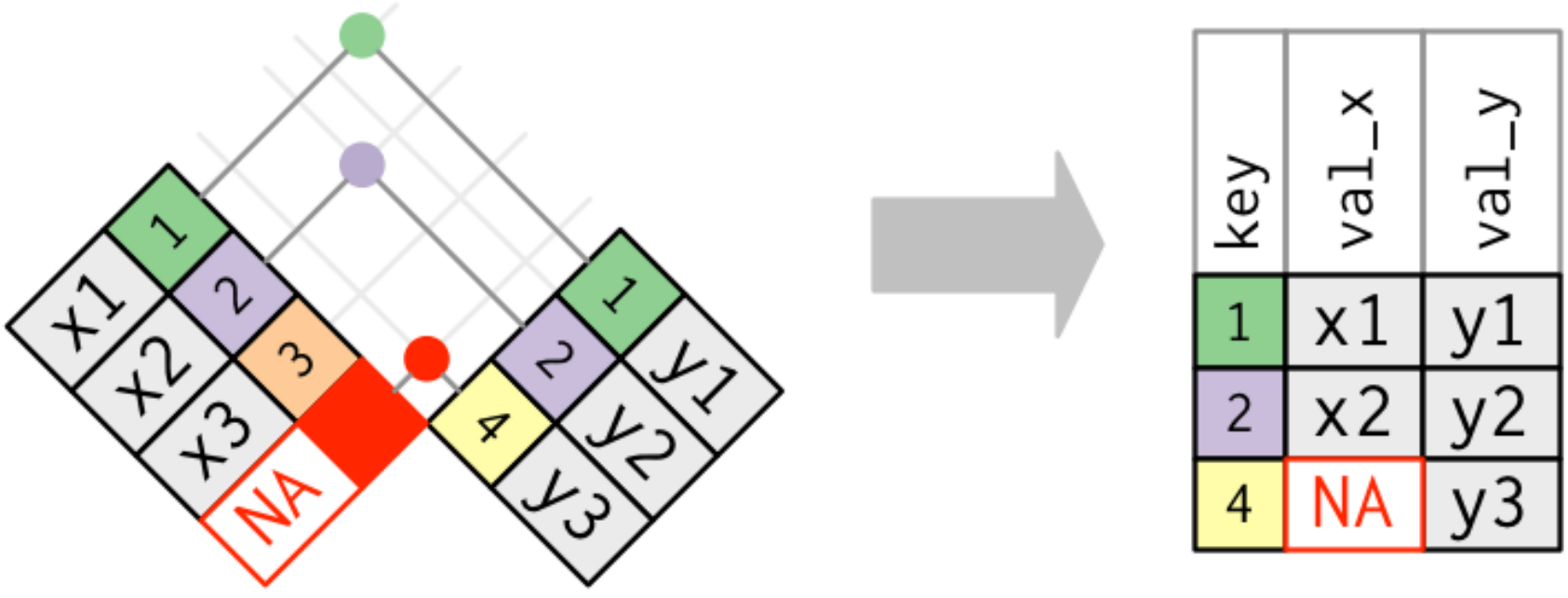
We're going to focus on four types of join:

`inner_join` matches pairs of observations whenever their keys are equal.

`left_join` keeps all observations in `x`.

`right_join` keeps all observations in `y`.

`full_join` keeps all observations in `x` and `y`.



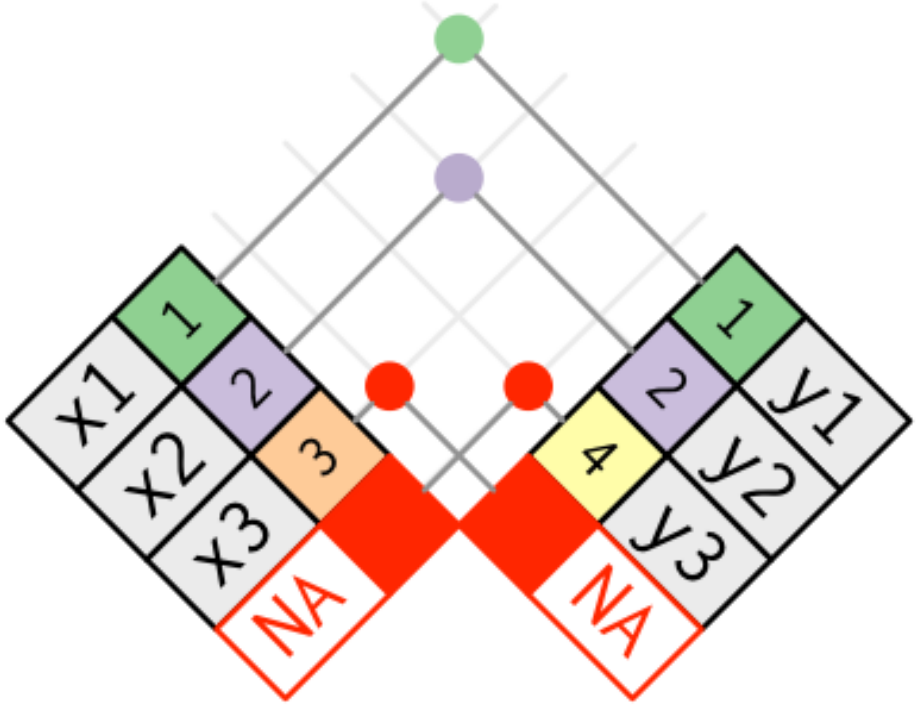
We're going to focus on four types of join:

`inner_join` matches pairs of observations whenever their keys are equal.

`left_join` keeps all observations in `x`.

`right_join` keeps all observations in `y`.

`full_join` keeps all observations in `x` and `y`.



key	val_x	val_y
1	x1	y1
2	x2	y2
3	x3	NA
4	NA	y3

- By default, data frames are joined based on variables that appear in both tables.
- Unlike other packages, you don't always need to specify the joining key.

  If in doubt, I try `full_join` first and drop matches that aren't needed.

- By default, data frames are joined based on variables that appear in both tables.
- Unlike other packages, you don't always need to specify the joining key.

  If in doubt, I try `full_join` first and drop matches that aren't needed.



`band_members`

name	band
Mick	Stones
John	Beatles
Paul	Beatles



`band_instruments`

name	plays
John	guitar
Paul	bass
Keith	guitar

```
# inner_join
band_members |> inner_join(band_instruments)
```

```
# inner_join
band_members |> inner_join(band_instruments)
```

```
# left_join
band_members |> left_join(band_instruments)
```

```
# inner_join
band_members |> inner_join(band_instruments)
```

```
# left_join
band_members |> left_join(band_instruments)
```

```
# right_join
band_members |> right_join(band_instruments)
```

```r
# inner_join
band_members |> inner_join(band_instruments)
```

```r
# left_join
band_members |> left_join(band_instruments)
```

```r
# right_join
band_members |> right_join(band_instruments)
```

```r
# full_join
band_members |> full_join(band_instruments)
```

Filtering joins

Filtering joins keep or discard rows in x depending on whether they have matches in y:

- `semi_join` keeps the rows in x that **do** have a match in y.
- `anti_join` keeps the rows in x that **do not** have a match in y.

More on filtering joins in the dplyr reference

Filtering joins

```
semi_join(
  band_members,
  band_instruments,
  by = "name"
)
# ---> Band members who HAVE instruments

anti_join(
  band_members,
  band_instruments,
  by = "name"
)
# ---> Band members with NO instrument
```

Tidy data

What is tidy data?

> *"All happy families resemble one another; every unhappy family is unhappy in its own way."*
>
> *Leo Tolstoy (1878)*

> *"Tidy datasets are all alike, but every messy dataset is messy in its own way."*
>
> *Hadley Wickham (2014)*



Journal of Statistical Software
MMMMMM YYYY, Volume VV, Issue II. *http://www.jstatsoft.org/*

Tidy Data

Hadley Wickham
RStudio

Abstract

A huge amount of effort is spent cleaning data to get it ready for analysis, but there has been little research on how to make data cleaning as easy and effective as possible. This paper tackles a small, but important, component of data cleaning: data tidying. Tidy datasets are easy to manipulate, model and visualise, and have a specific structure: each variable is a column, each observation is a row, and each type of observational unit is a table. This framework makes it easy to tidy messy datasets because only a small set of tools are needed to deal with a wide range of un-tidy datasets. This structure also makes it easier to develop tidy tools for data analysis, tools that both input and output tidy datasets. The advantages of a consistent data structure and matching tools are demonstrated with a case study free from mundane data manipulation chores.

Keywords: data cleaning, data tidying, relational databases, R.

1. Introduction

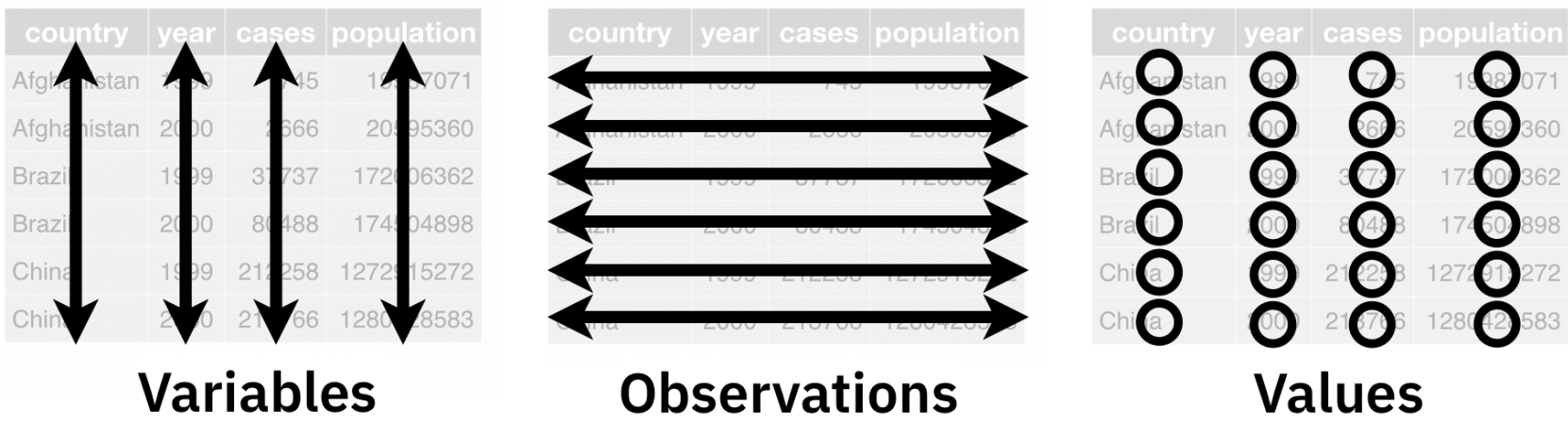
It is often said that 80% of data analysis is spent on the process of cleaning and preparing the data (Dasu and Johnson 2003). Data preparation is not just a first step, but must be repeated many over the course of analysis as new problems come to light or new data is collected. Despite the amount of time it takes, there has been surprisingly little research on how to clean data well. Part of the challenge is the breadth of activities it encompasses: from outlier checking, to date parsing, to missing value imputation. To get a handle on the problem, this paper focusses on a small, but important, aspect of data cleaning that I call data **tidying**: structuring datasets to facilitate analysis.

The principles of tidy data provide a standard way to organise data values within a dataset. A standard makes initial data cleaning easier because you don't need to start from scratch and reinvent the wheel every time. The tidy data standard has been designed to facilitate initial exploration and analysis of the data, and to simplify the development of data analysis tools that work well together. Current tools often require translation. You have to spend time

Journal of Statistical Software, **59**(10), 1–23.
https://doi.org/10.18637/jss.v059.i10



1. Each variable forms a column.
2. Each observation forms a row.
3. Each value is a cell.



A **variable** is something you measure (country, year, cases). An **observation** is everything measured on one unit (one country in one year).

Same data, two layouts

Untidy

```
# A tibble: 3 x 3
  country     `1999` `2000`
  <chr>        <dbl>  <dbl>
1 Afghanistan    745   2666
2 Brazil       37737  80488
3 China       212258 213766
```

Tidy

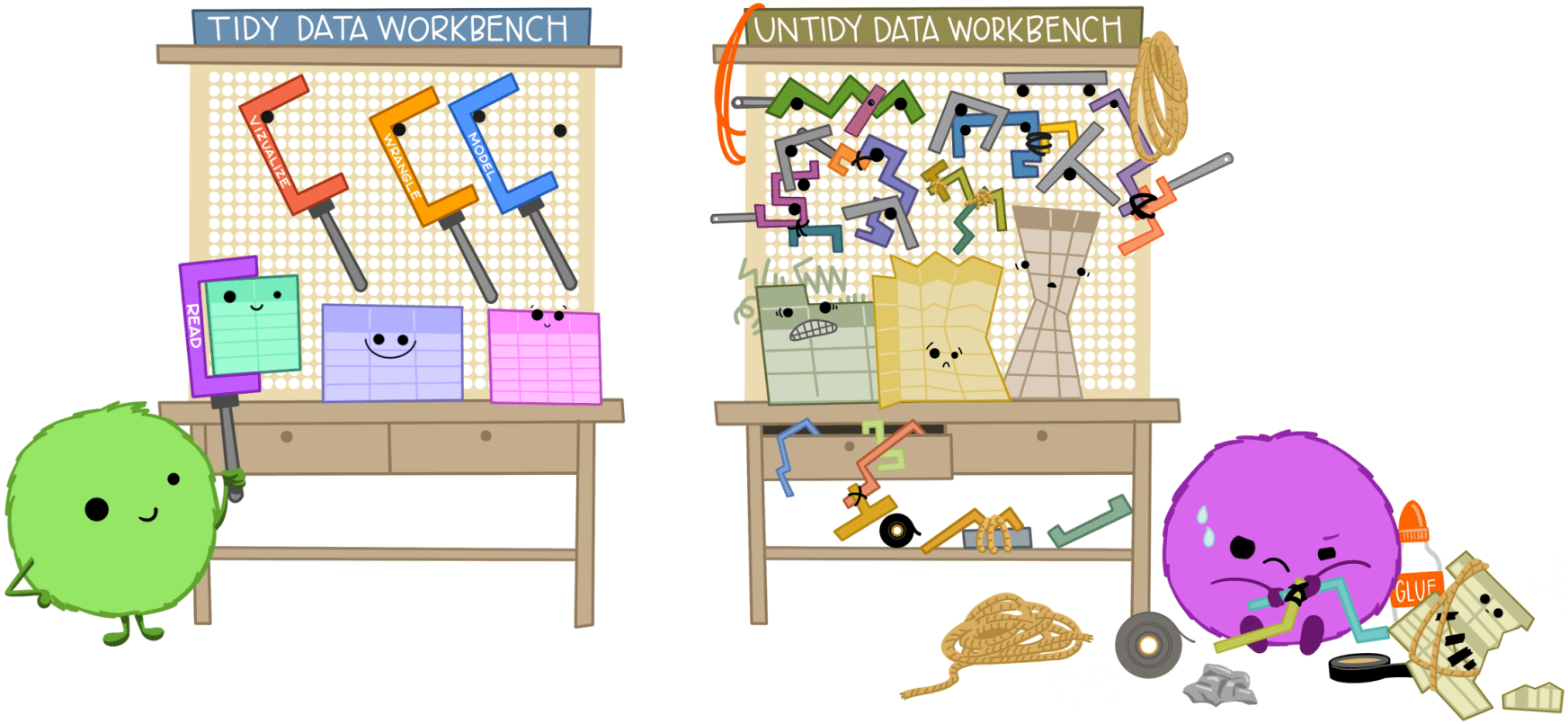
```
# A tibble: 6 x 3
  country      year  cases
  <chr>       <dbl>  <dbl>
1 Afghanistan  1999    745
2 Afghanistan  2000   2666
3 Brazil       1999  37737
4 Brazil       2000  80488
5 China        1999 212258
6 China        2000 213766
```

- On the left, 1999 and 2000 are values of a variable (year), stored as column names.
- Column names that look like data (years, weeks, income bands) are the most common sign of untidy data. `pivot_longer` fixes this.

`tidyr::table4a` and `tidyr::table1`

When working with tidy data, we can use the same tools in similar ways for different datasets...

...but working with untidy data often means reinventing the wheel with one-time approaches that are hard to iterate or reuse.



Illustrations adapted from the Openscapes blog *Tidy Data for reproducibility, efficiency, and collaboration* by Julia Lowndes and Allison Horst.

a) Select columns or rows

b) Sorting a dataset

c) Creating or modifying columns

d) Combining datasets

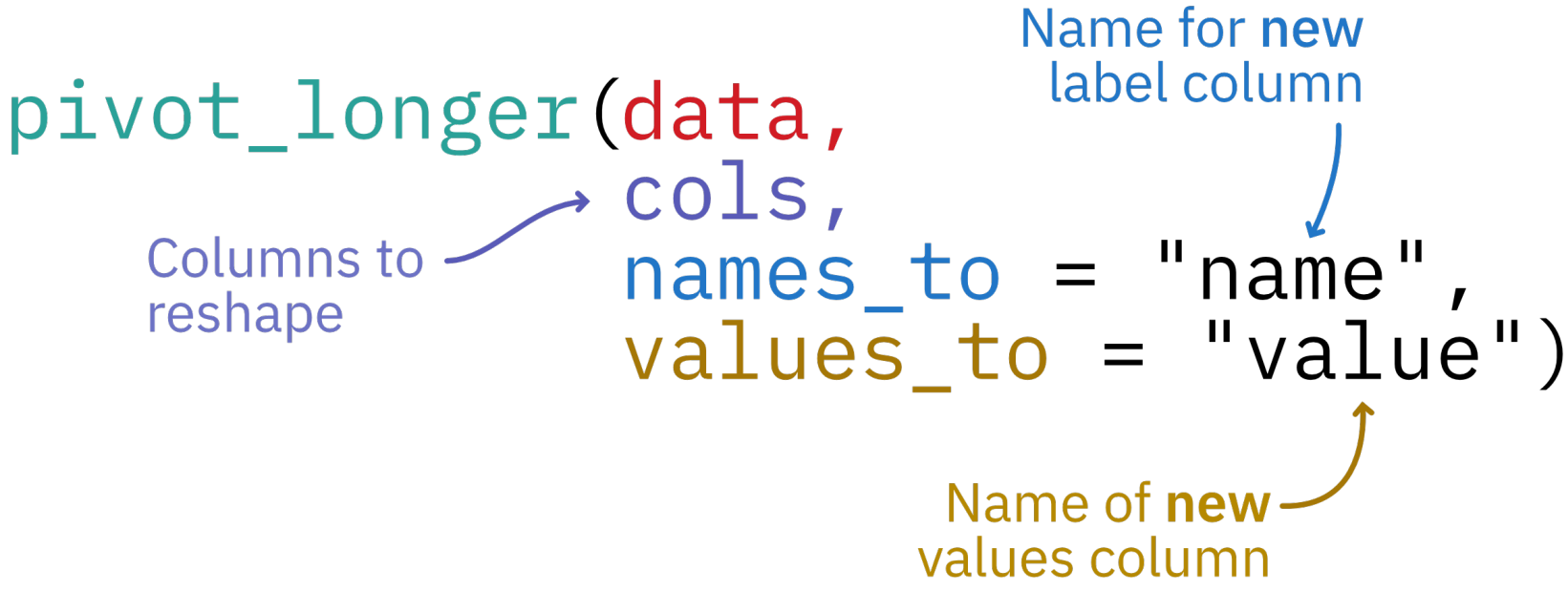
e) Reshaping a dataset

f) Grouping and summarising data

e) From WIDE to LONG with `pivot_longer`

```
> relig_income
# A tibble: 18 x 11
   religion `<$10k` `$10-20k` `$20-30k` `$30-40k` `$40-50k`
   <chr>      <dbl>     <dbl>     <dbl>     <dbl>     <dbl>
 1 Agnostic      27        34        60        81        76
 2 Atheist       12        27        37        52        35
 3 Buddhist      27        21        30        34        33
 4 Catholic     418       617       732       670       638
 5 Don't know    15        14        15        11        10
 6 Evangel...   575       869      1064       982       881
 7 Hindu          1         9         7         9        11
 8 Histori...   228       244       236       238       197
 9 Jehovah       20        27        24        24        21
10 Jewish        19        19        25        25        30
11 Mainlin      289       495       619       655       651
12 Mormon        29        40        48        51        56
13 Muslim         6         7         9        10         9
14 Orthodox      13        17        23        32        32
15 Other C...     9         7        11        13        13
16 Other F...    20        33        40        46        49
17 Other W...     5         2         3         4         2
18 Unaffil...   217       299       374       365       341
# i 5 more variables: `$50-75k` <dbl>, `$75-100k` <dbl>,
#   `$100-150k` <dbl>, `>150k` <dbl>, `Don't know/refused` <dbl>
```

From WIDE to LONG with pivot_longer



```
> relig_income
# A tibble: 18 x 11
   religion `<$10k` `$10-20k` `$20-30k` `$30-40k` `$40-50k`
   <chr>      <dbl>     <dbl>     <dbl>     <dbl>     <dbl>
 1 Agnostic      27        34        60        81        76
 2 Atheist       12        27        37        52        35
 3 Buddhist      27        21        30        34        33
 4 Catholic     418       617       732       670       638
 5 Don't know    15        14        15        11        10
 6 Evangel...   575       869      1064       982       881
 7 Hindu          1         9         7         9        11
 8 Histori...   228       244       236       238       197
 9 Jehovah       20        27        24        24        21
10 Jewish        19        19        25        25        30
11 Mainlin      289       495       619       655       651
12 Mormon        29        40        48        51        56
13 Muslim         6         7         9        10         9
14 Orthodox      13        17        23        32        32
15 Other C...     9         7        11        13        13
16 Other F...    20        33        40        46        49
17 Other W...     5         2         3         4         2
18 Unaffil...   217       299       374       365       341
# i 5 more variables: `$50-75k` <dbl>, `$75-100k` <dbl>,
#   `$100-150k` <dbl>, `>150k` <dbl>, `Don't know/refused` <dbl>
```

```r
relig_income |>
    pivot_longer(cols = -religion,
                 names_to = "income",
                 values_to = "count")
```

```
# A tibble: 180 x 3
   religion                income                count
   <chr>                   <chr>                 <dbl>
 1 Agnostic                <$10k                    27
 2 Agnostic                $10-20k                  34
 3 Agnostic                $20-30k                  60
 4 Agnostic                $30-40k                  81
 5 Agnostic                $40-50k                  76
 6 Agnostic                $50-75k                 137
 7 Agnostic                $75-100k                122
 8 Agnostic                $100-150k               109
 9 Agnostic                >150k                    84
10 Agnostic                Don't know/refused       96
11 Atheist                 <$10k                    12
# i 169 more rows
```

Another example...

```
> billboard
        artist                    track date.entered wk1 wk2 wk3 wk4
         2 Pac Baby Don't Cry (Keep...   2000-02-26  87  82  72  77
       2Ge+her The Hardest Part Of ...   2000-09-02  91  87  92  NA
  3 Doors Down               Kryptonite   2000-04-08  81  70  68  67
  3 Doors Down                    Loser   2000-10-21  76  76  72  69
      504 Boyz             Wobble Wobble   2000-04-15  57  34  25  17
          98^0 Give Me Just One Nig...   2000-08-19  51  39  34  26
       A*Teens            Dancing Queen   2000-07-08  97  97  96  95
       Aaliyah            I Don't Wanna   2000-01-29  84  62  51  41
       Aaliyah                Try Again   2000-03-18  59  53  38  28
Adams, Yolanda            Open My Heart   2000-08-26  76  76  74  69
 Adkins, Trace                     More   2000-04-29  84  84  75  73
  Alice Deejay         Better Off Alone   2000-04-08  79  65  53  48
   Allan, Gary Smoke Rings In The D...   2000-01-22  80  78  76  77
         Amber                   Sexual   1999-07-17  99  99  96  96
     Anastacia           I'm Outta Love   2000-04-01  92  NA  NA  95
 Anthony, Marc              My Baby You   2000-09-16  82  76  76  70
 Anthony, Marc           You Sang To Me   2000-02-26  77  54  50  43
         Avant            My First Love   2000-11-04  70  62  56  43
         Avant                Separated   2000-04-29  62  32  30  23
# ... columns wk5 to wk76 not shown
```

```
> billboard |>
+   pivot_longer(starts_with("wk"),
+                names_to = "week",
+                values_to = "chart_position")
 # A tibble: 24,092 × 5
   artist track                   date.entered week  chart_position
   <chr>  <chr>                   <date>       <chr>          <dbl>
 1 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk1               87
 2 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk2               82
 3 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk3               72
 4 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk4               77
 5 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk5               87
 6 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk6               94
 7 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk7               99
 8 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk8               NA
 9 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk9               NA
10 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk10              NA
```

```
> billboard |>
+   pivot_longer(starts_with("wk"),
+                names_to = "week",
+                values_to = "chart_position") |>
+   mutate(week = parse_number(week))
# A tibble: 24,092 × 5
   artist track                   date.entered  week chart_position
   <chr>  <chr>                   <date>       <dbl>          <dbl>
 1 2 Pac  Baby Don't Cry (Keep... 2000-02-26       1             87
 2 2 Pac  Baby Don't Cry (Keep... 2000-02-26       2             82
 3 2 Pac  Baby Don't Cry (Keep... 2000-02-26       3             72
 4 2 Pac  Baby Don't Cry (Keep... 2000-02-26       4             77
 5 2 Pac  Baby Don't Cry (Keep... 2000-02-26       5             87
 6 2 Pac  Baby Don't Cry (Keep... 2000-02-26       6             94
 7 2 Pac  Baby Don't Cry (Keep... 2000-02-26       7             99
 8 2 Pac  Baby Don't Cry (Keep... 2000-02-26       8             NA
 9 2 Pac  Baby Don't Cry (Keep... 2000-02-26       9             NA
10 2 Pac  Baby Don't Cry (Keep... 2000-02-26      10             NA
```

```
> billboard |>
+   pivot_longer(starts_with("wk"),
+                names_to = "week",
+                values_to = "chart_position",
+                values_drop_na = TRUE) |>
+   mutate(week = parse_number(week))
 # A tibble: 5,307 × 5
   artist  track                   date.entered  week chart_position
   <chr>   <chr>                   <date>       <dbl>          <dbl>
 1 2 Pac   Baby Don't Cry (Keep... 2000-02-26       1             87
 2 2 Pac   Baby Don't Cry (Keep... 2000-02-26       2             82
 3 2 Pac   Baby Don't Cry (Keep... 2000-02-26       3             72
 4 2 Pac   Baby Don't Cry (Keep... 2000-02-26       4             77
 5 2 Pac   Baby Don't Cry (Keep... 2000-02-26       5             87
 6 2 Pac   Baby Don't Cry (Keep... 2000-02-26       6             94
 7 2 Pac   Baby Don't Cry (Keep... 2000-02-26       7             99
 8 2Ge+her The Hardest Part Of ... 2000-09-02       1             91
 9 2Ge+her The Hardest Part Of ... 2000-09-02       2             87
10 2Ge+her The Hardest Part Of ... 2000-09-02       3             92
```

```
> longer <- billboard |>
+   pivot_longer(starts_with("wk"),
+                names_to = "week",
+                values_to = "chart_position",
+                values_drop_na = TRUE) |>
+   mutate(week = parse_number(week))
```

From LONG to WIDE with `pivot_wider`

```
pivot_wider(data,
            names_from = "name",
            values_from = "value")
```

An **existing** column that will become the column headings

An **existing** column containing the values.

Suppose we already have our data in LONG format...

artist	track	date.entered	week	chart_position
2 Pac	Baby Don't	2000-02-26	1	87
2 Pac	Baby Don't	2000-02-26	2	82
2 Pac	Baby Don't	2000-02-26	3	72
2 Pac	Baby Don't	2000-02-26	4	77
2 Pac	Baby Don't	2000-02-26	5	87
2 Pac	Baby Don't	2000-02-26	6	94
2 Pac	Baby Don't	2000-02-26	7	99
2Ge+her	The Hardest	2000-09-02	1	91
2Ge+her	The Hardest	2000-09-02	2	87
2Ge+her	The Hardest	2000-09-02	3	92
3 Doors Down	Kryptonite	2000-04-08	1	81
3 Doors Down	Kryptonite	2000-04-08	2	70
3 Doors Down	Kryptonite	2000-04-08	3	68
3 Doors Down	Kryptonite	2000-04-08	4	67
3 Doors Down	Kryptonite	2000-04-08	5	66
3 Doors Down	Kryptonite	2000-04-08	6	57
3 Doors Down	Kryptonite	2000-04-08	7	54
3 Doors Down	Kryptonite	2000-04-08	8	53
3 Doors Down	Kryptonite	2000-04-08	9	51
3 Doors Down	Kryptonite	2000-04-08	10	51

```
pivot_wider(data,
            names_from = ?,
            values_from = ?)
```

pivot_wider

```
pivot_wider(names_from = NEW COLUMN NAMES,
            values_from = VALUES)
```

```
> longer |>
+   select(artist, track, week, chart_position) |>
+   pivot_wider(names_from = week,
+               values_from = chart_position,
+               names_prefix = "wk")
# A tibble: 317 x 67
  artist   track   wk1   wk2   wk3   wk4   wk5   wk6   wk7   wk8
  <chr>    <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 2 Pac    Baby~    87    82    72    77    87    94    99    NA
2 2Ge+her  The ~    91    87    92    NA    NA    NA    NA    NA
3 3 Doors~ Kryp~    81    70    68    67    66    57    54    53
# i 314 more rows
```

`names_prefix` adds text to the start of each new column name.

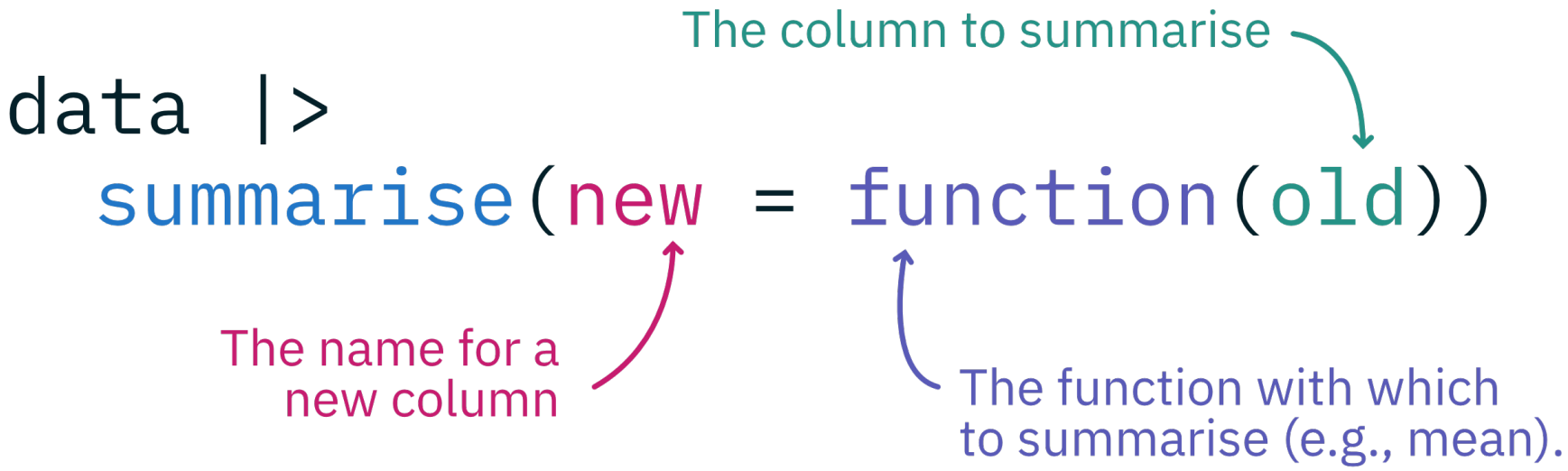
Practical: pivoting between LONG and WIDE

1. Reshape the `fish_encounters` dataset to WIDE format, such that each column represents a different monitoring station. After reshaping, is this dataset 'tidy'? Why?

2. Reshape the `world_bank_pop` dataset to LONG format, such that it contains four columns: `country`, `indicator`, `year`, and `value`.

3. (If time) Reshape the `table2` dataset such that there is a separate column for 'cases' and 'population'.

f) Grouping and summarising data



Note the differences with `mutate`

`mutate`

1 row = 1 row



`summarise`

Many rows = 1 row



For example, calculate the mean and standard deviation of a column:

```
> mtcars |>
+   summarise(mean = mean(wt),
+             sd = sd(wt))
     mean        sd
1 3.21725 0.9784574
```

As with mutate, we can have multiple expressions, separated by commas.

Grouping data with `group_by`

We often want to calculate summaries for subgroups in our data.

```
> # Average fuel efficiency by number
> # of cylinders?
> mtcars |>
+   group_by(cyl) |>
+   summarise(efficiency = mean(mpg))
# A tibble: 3 x 2
    cyl efficiency
  <dbl>      <dbl>
1     4       26.7
2     6       19.7
3     8       15.1
```

Note that, once you define the grouping, all subsequent operations are grouped.

For example, `mutate`:

```
> mtcars |>
+   group_by(cyl) |>
+   mutate(max = max(mpg))
```

This will calculate the maximum per group.

Counting rows with `count`

`count()` is a shortcut for `group_by()` then `summarise(n = n())`:

```
> admissions |>
+   count(provider_code)
# A tibble: 7 x 2
  provider_code     n
  <chr>         <int>
1 R11             982
2 R12            1044
3 R13             974
4 R14            1076
5 R15             955
6 R16             880
7 R17             800
```

```
> admissions |>
+   count(provider_code,
+         sort = TRUE)
# A tibble: 7 x 2
  provider_code     n
  <chr>         <int>
1 R14            1076
2 R12            1044
3 R11             982
4 R13             974
5 R15             955
6 R16             880
7 R17             800
```

n and n_distinct

Inside `summarise()`: `n()` counts rows; `n_distinct()` counts different values.

```
> admissions |>
+   group_by(provider_code) |>
+   summarise(spells = n(),
+             patients = n_distinct(patient_id),
+             mean_los = mean(los_days))
# A tibble: 7 x 4
  provider_code spells patients mean_los
  <chr>          <int>    <int>    <dbl>
1 R11              982      943     10.6
2 R12             1044     1003     11.2
3 R13              974      934     11.8
4 R14             1076     1028     12.0
5 R15              955      909     12.4
6 R16              880      845     12.8
7 R17              800      772     13.4
```

Other useful verbs

```
# Unique combinations of values
admissions |>
  distinct(provider_code, main_specialty)

# Rows by position, at random, or largest values
admissions |> slice_head(n = 5)
admissions |> slice_sample(n = 100)
admissions |> slice_max(los_days, n = 3)

# Move columns to the front
admissions |> relocate(patient_id, los_days)

# Extract one column as a vector
admissions |> pull(los_days)
```

Phew, that was a lot...

Essential data manipulation tasks

a) Select columns or rows
b) Sorting a dataset
c) Creating or modifying columns
d) Combining datasets
e) Reshaping a dataset
f) Grouping and summarising data